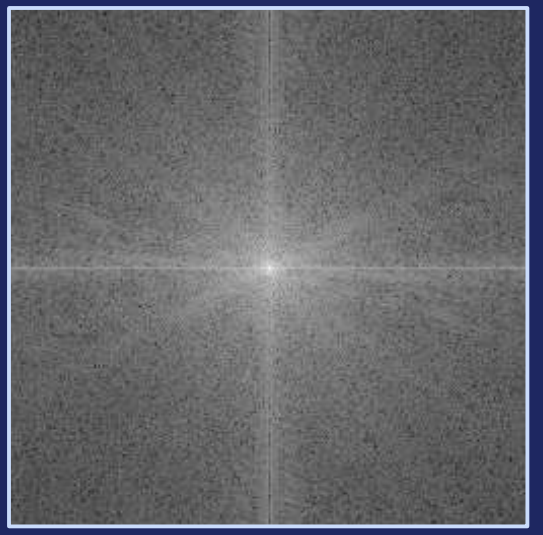
IMAGE PROCESSING

Frequency-Domain Processing

with the Discrete Fourier Transform



magnitude spectrum



original image

THE BIG IDEA

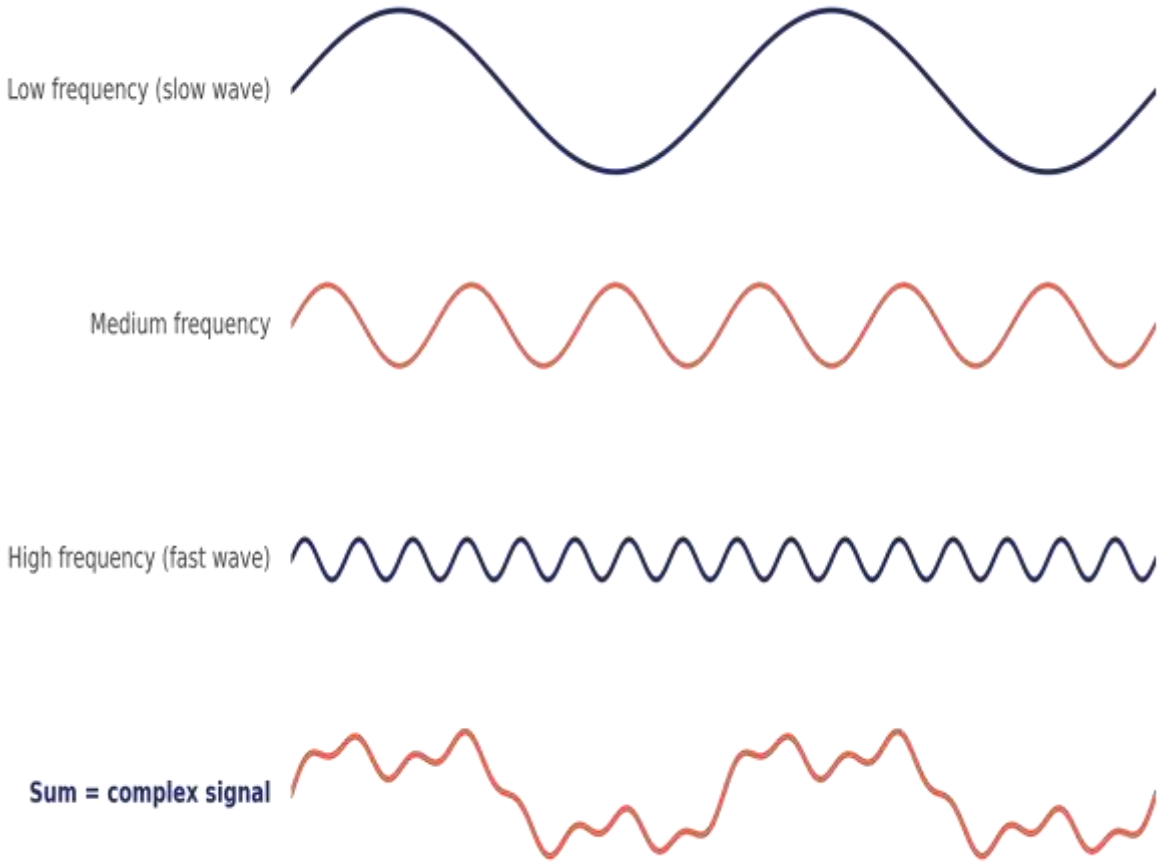
Two Ways to Look at an Image

Spatial domain: the image as we normally see it — a grid of pixel brightness values.

Frequency domain: the same image described as a mix of waves — how fast brightness changes across the image.

Simple way to think about it:

Any image — no matter how complex — can be rebuilt by adding together many simple sine/cosine waves of different frequencies, orientations, and strengths. The DFT finds those waves.



$$F(u, v) = \Sigma\Sigma\ f(x, y)\ \cdot\ e^{(-j2\pi(ux/M + vy/N))}$$

$f(x, y)$	Pixel intensity at spatial location (x, y)
M, N	Width and height of the image
u, v	Frequency being tested (horizontal, vertical)
$e^{(-j2\pi...)}$	A complex sinusoidal basis wave being tested

In plain words: for each candidate frequency (u, v), multiply every pixel by a wave oscillating at that frequency and sum the results.

03

DFT Output & OpenCV Implementation

F(u,v) is complex-valued

Magnitude — how strongly a frequency is present

Phase — where those patterns are positioned; carries most structural information

fftshift()

Reorders the spectrum so low frequencies (DC) sit at the center and high frequencies spread outward — for visualization and filtering convenience.

Python — OpenCV

```
import cv2, numpy as np

img = cv2.imread('image.jpg',
    cv2.IMREAD_GRAYSCALE)

dft = cv2.dft(np.float32(img),
    flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)

mag = 20*np.log(cv2.magnitude(
    dft_shift[:,:,0],
    dft_shift[:,:,1]) + 1)
```

THE BASICS

What Does the DFT Actually Do?

The Discrete Fourier Transform converts an image from the spatial domain into the frequency domain — and back again.

Input

A grid of pixel brightness values (the image you see).

→

DFT

Decomposes the image into sine/cosine waves of different frequencies.

→

Output

A spectrum: how much of each frequency is present, and in which direction.

→

Inverse DFT

Adds the waves back together to reconstruct the image (or a filtered version).

No pixel values are lost — the DFT is just a different, reversible way of describing the same image.

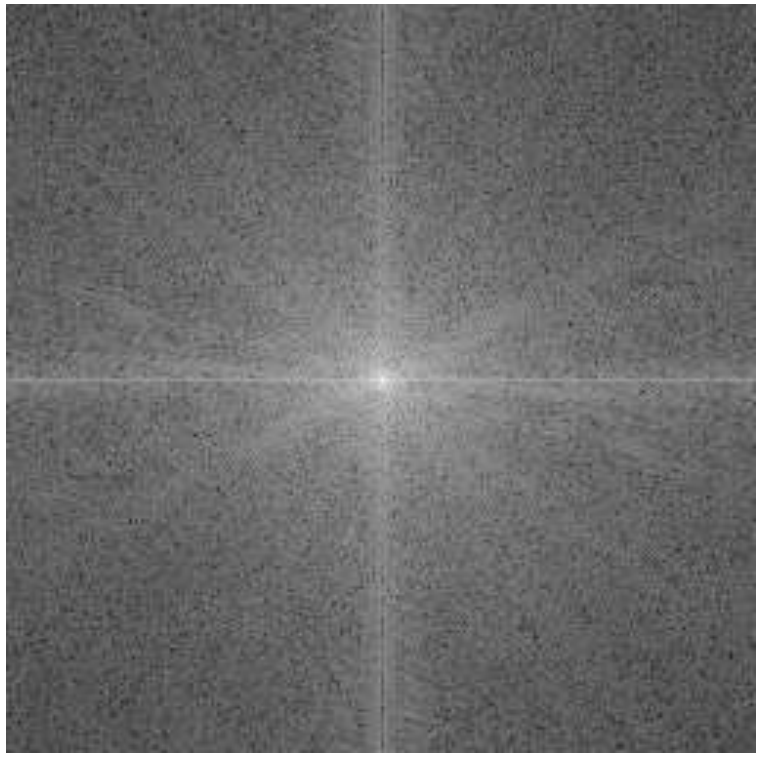
SEE IT IN ACTION

The Magnitude Spectrum



Original Image

DFT →



Magnitude Spectrum

Reading it:

- Bright center = strong low frequencies (overall brightness, broad shapes)
- Bright streaks = strong edges/lines in that orientation
- Dim outer areas = little fine detail/noise

CORE CONCEPT

What Low & High Frequencies Mean



LOW FREQUENCY

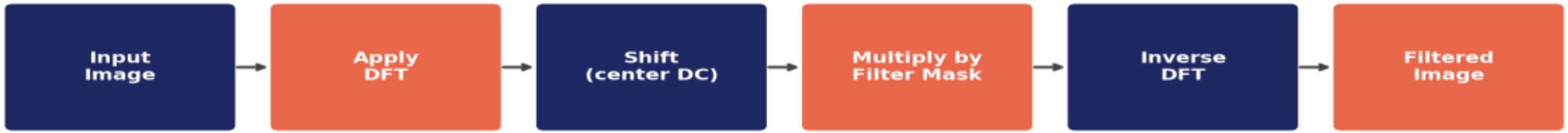
Slow, smooth changes across the image — overall brightness, broad shapes, gradual shading. Near the center of the spectrum.

HIGH FREQUENCY

Fast, abrupt changes — edges, fine texture, sharp detail, and noise. Near the outer edges of the spectrum.

HOW FILTERING WORKS

The Frequency-Domain Filtering Pipeline



The filter is just a mask multiplied onto the spectrum: keep the frequencies you want, zero out the rest.

Low-pass mask → keeps center (blur) *High-pass mask → keeps edges (sharpen/detect edges)*

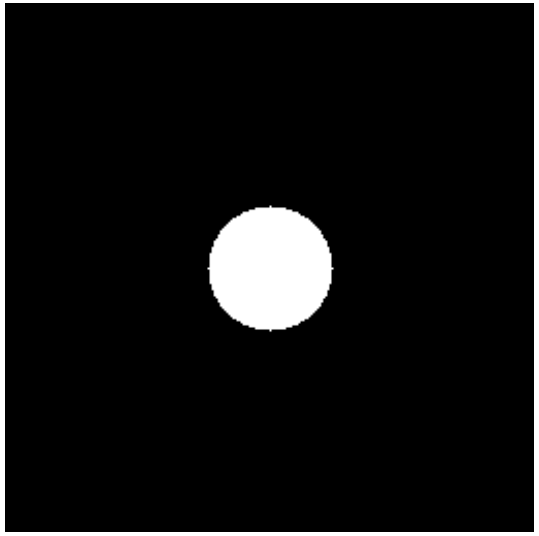
FILTERING EXAMPLE — 1 OF 2

Low-Pass Filtering = Blurring

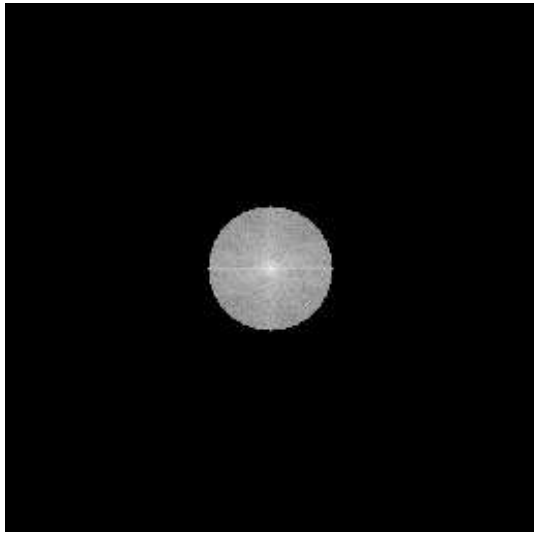
Keep only the center of the spectrum (low frequencies) and zero out the rest:



Original



Low-Pass Mask



Filtered Spectrum



Result (Blurred)

Sharp mask edges cause faint ripple artifacts (“ringing”) — a smoother mask (e.g. Gaussian) avoids this.

FILTERING EXAMPLE — 2 OF 2

High-Pass Filtering = Edge Detection

Keep only the outer spectrum (high frequencies) and zero out the center:



Original



High-Pass Mask



Result (Edges)

Removing low frequencies removes smooth shading — only fast brightness changes (edges) survive.

This is the frequency-domain equivalent of a spatial sharpening/edge mask (like the Laplacian).

TUNING THE FILTER

Radius Controls the Strength

A smaller low-pass radius keeps fewer frequencies — more blur:



Radius = 10
(very blurred)



Radius = 30
(moderate)



Radius = 60
(mostly sharp)

Smaller radius = fewer frequencies kept = more information removed = stronger blur.

WHY BOTHER?

Noise removal

Periodic noise shows up as isolated bright spots in the spectrum — easy to remove.

Compression

JPEG discards less-important high frequencies most viewers won't notice.

Blurring & sharpening

Fast, tunable smoothing or detail enhancement via simple masks.

Pattern & texture analysis

Repeating patterns create distinctive, identifiable spectral signatures.

WRAP UP

Key Takeaways

1. The DFT rewrites an image as a mix of waves of different frequencies — fully reversible.
2. The magnitude spectrum shows how strong each frequency is; center = low, edges = high.
3. Low frequencies = smooth/overall shape; high frequencies = edges, detail, and noise.
4. Filtering in the frequency domain = multiply the spectrum by a mask, then inverse-DFT back.
5. Low-pass masks blur; high-pass masks sharpen or reveal edges.
6. Smooth masks (Gaussian) avoid the ringing artifacts sharp-edged masks can introduce.

PART 2

XML and YAML in OpenCV File Handling

06

Why XML and YAML in OpenCV?

OpenCV needs to save and load structured, non-image data — camera calibration matrices, classifier parameters, or configuration. The FileStorage class reads/writes this data as XML, YAML, or JSON; the same API works for all three, only the file extension changes.

Human-readable

Open and inspect/edit the file directly

Structured / hierarchical

Naturally supports nested data

Cross-platform

Same file loads in C++, Python, Java bindings

Native type support

cv::Mat and sequences serialize automatically

File Handling Cycle: open → read/parse → build in-memory objects → access/modify → write back

07

XML File Handling

employees.xml

```
<employee id="E101">
  <name>Anita</name>
  <department>Sales</department>
  <salary>55000</salary>
</employee>
```

```
import xml.etree.ElementTree as ET

tree = ET.parse('employees.xml')
root = tree.getroot()

for emp in root.findall('employee'):
    name = emp.find('name').text
```

Tags & Attributes

Explicit, hierarchical, document-like. Each data item sits inside named tags; attributes attach directly to a tag (e.g. id="E101").

Real-World XML Use

- Banking / ERP invoice & transaction exchange
- DOCX / XLSX internal structure
- SOAP web service messages
- Catalog & metadata systems

08

YAML File Handling

config.yaml

```
server:
  host: localhost
  port: 8000
settings:
  debug: true
  log_level: info
```

```
import yaml

with open('config.yaml') as f:
    config = yaml.safe_load(f)

config['settings']['debug'] = False
yaml.dump(config, open('out.yaml','w'))
```

Indentation, Not Tags

Shorter, cleaner, and easier for humans to read and edit. Structure comes from indentation and key-value pairs rather than opening/closing tags.

Real-World YAML Use

- Docker Compose services, ports, volumes
- Kubernetes deployment manifests
- CI/CD pipeline definitions
- ML hyperparameters (lr, batch size, epochs)

09

XML vs. YAML — Choosing & Best Practices

Aspect	XML	YAML
Structure	Tags & attributes	Indentation, key-value
Readability	More verbose	Cleaner, concise
Best fit	Document / enterprise integration	Configuration files

Parameter Persistence

Saving settings so they remain available after the application closes — without it, users must re-enter values every run.

Key Takeaways

1. The DFT decomposes an image into frequency components, enabling low-pass, high-pass, and notch filtering via simple multiplication in the frequency domain.

2. The FFT is the fast algorithm that makes computing the DFT practical on real image sizes.

3. OpenCV's FileStorage class reads/writes XML and YAML through one shared API — XML for document/enterprise data, YAML for readable configuration and persistence.